

The Inspector Column: Editable Code, Typing, and Outline

Implementation of Block-MiniJava's Three Program-View Tabs

Research note — Block-MiniJava

2026-07-20

Abstract

The inspector column (`<aside id="code-column">`, `src/index.html:210--238`) is Block-MiniJava's dedicated "view the program as something other than blocks" panel. It hosts three tabs that all read the *same* live workspace and never own a separate copy of the program: **Editable Code** (a bidirectional MiniJava text $\leftrightarrow$ blocks editor), **Typing** (the on-demand natural-deduction derivation viewer documented separately in `typing-and-evaluation-rules.tex`), and **Outline** (a collapsible tree of the block program's structure). This note explains how each tab is implemented and kept synchronized with workspace edits, with every mechanism traced to its file and line.

Contents

1	The inspector column and its tab switch	1
2	Editable Code	2
2.1	Blocks $\rightarrow$ text	2
2.2	The overlay DOM: line numbers, syntax color, and a transparent textarea	3
2.3	The import pipeline: debounce, parse, validate	3
2.4	The structural-equality guard: never delete what the user just typed	3
2.5	Breaking the sync feedback loop	4
2.6	Other editor entry points	4
3	Typing	4
4	Outline	5
4.1	Rendering	5
4.2	Click-to-locate	6
4.3	Why Outline needs no staleness guard	6
5	Cross-cutting: the provenance idiom	6
	Note on scope	7

1 The inspector column and its tab switch

Element	Role
<code>.inspector-tabs</code>	<code>index.html:219--223</code> — three <code><button role="tab"></code> s: <code>tab-code</code> , <code>tab-typing</code> , <code>tab-outline</code> , each with <code>data-panel</code> naming the panel it controls.

Element	Role
<code>.inspector-panel</code>	<code>index.html:225--236</code> — three <code><section role="tabpanel"></code> s: <code>panel-code</code> , <code>panel-typing</code> , <code>panel-outline</code> ; the active one carries class <code>is-active</code> .
<code>InspectorPanel</code> type	<code>app.ts:70</code> — <code>'code' 'typing' 'outline'</code> , the closed set <code>selectInspectorPanel</code> dispatches on.
<code>selectInspectorPanel(panel)</code>	<code>app.ts:353--366</code> — the one function every tab click routes through: toggles <code>is-active/aria-selected</code> on the matching tab and panel pair, then triggers that panel's own on-activate render (<code>scheduleOutlineRender</code> for <code>'outline'</code> , <code>scheduleTypingRender</code> for <code>'typing'</code> — Editable Code needs no such call, since it stays synchronized continuously, §2), and finally calls <code>requestLayoutResize(false)</code> since a newly-shown panel can change scrollbar/wrap width.

Only *Typing* and *Outline* defer their rendering to tab activation (`scheduleTypingRender/scheduleOutlineRender` both `return` immediately if their panel is not the active one — see `typingPanel.ts:163--166` and §4.1 — this is a cheap no-op check against needlessly rebuilding an invisible DOM tree). **Editable Code** keeps no such guard: its textarea is the one surface a user can type into at any time, so it must always reflect (or be reflected onto) the workspace regardless of which tab is currently showing.

2 Editable Code

`src/core/ui/codeEditor.ts` (307 lines) implements a full bidirectional editor: `blocks → text` is a pure, always-available function (`generateMiniJava`); `text → blocks` is a debounced, validated import that only ever fires from a real keystroke, never from a workspace-originated change, so the two directions cannot feed each other in a loop.

2.1 Blocks → text

Function	Role
<code>generateMiniJava(workspace)</code>	<code>core/generator/minijavaGenerator.ts</code> (271 lines) — walks the block tree from the <code>mj_goal</code> block and emits MiniJava source text; the single source of truth both the read-only highlighted view and the editable textarea are kept in sync with.
<code>highlightMiniJava(code)</code>	<code>core/generator/highlighter.ts</code> (193 lines) — a small MiniJava syntax highlighter (keywords, literals, comments) that renders the <i>overlay</i> layer behind the transparent textarea (§2.2).
<code>syncEditorFromWorkspace(workspace, code, message)</code>	<code>codeEditor.ts:94--112</code> — called by <code>app.ts</code> 's <code>updateCode()</code> (<code>app.ts:212--219</code>) after every workspace mutation. Two guards make it safe to call unconditionally: it is a no-op while the user is actively focused in the textarea (<code>document.activeElement === editor</code> , l.104), and it never overwrites text that already <i>means</i> the same program (§2.4).

2.2 The overlay DOM: line numbers, syntax color, and a transparent textarea

`createEditorDom()` (`codeEditor.ts:218--258`) builds the editor lazily, the first time `installEditableMiniJavaCodeEditor` runs, by rearranging the read-only `<pre id="generated-code">` element (`index.html:226`) that a headless/no-JS fallback would otherwise just display statically:

1. A new `.code-editor-pane` wraps a `.code-editor-line-numbers` `<pre>` (l.232–235, rebuilt by `renderLineNumbers`, l.56–60) and the original `<pre class="code-view">` — the latter is repurposed as the *syntax-highlight* layer, not removed.
2. A real `<textarea id="generated-code-editor">` is stacked *on top* of the highlight `<pre>` (l.238–246): the textarea is what the user actually types into and is visually transparent, while the `<pre>` underneath (kept in sync by `renderHighlight`, l.70–79) supplies the colored text the user sees through it.
3. `syncHighlightScroll/syncHighlightMetrics` (l.50–54, l.64–68) keep the highlight layer’s scroll offset and right-edge padding pixel-identical to the textarea’s, including compensating for whichever browser’s scrollbar happens to be present — a `ResizeObserver` (l.271–276) re-syncs on any panel resize.

This “transparent textarea over a colored `<pre>`” construction is the standard technique for a lightweight syntax-highlighting editor without a full editor component (no CodeMirror/Monaco dependency): the browser’s native textarea supplies cursor, selection, undo, and IME handling for free, and only the *visual* rendering is swapped out underneath it.

2.3 The import pipeline: debounce, parse, validate

Function	Role
<code>scheduleEditorImport(workspace, options)</code>	<code>codeEditor.ts:191--200</code> — fired on every <code>input</code> event (wired at l.266). Re-renders the highlight layer and status text immediately (so the view never looks stale), then debounces the actual import by <code>TEXT_IMPORT_DELAY_MS = 650ms</code> (l.6) via <code>window.setTimeout</code> , restarting the timer on every keystroke — so a workspace rebuild only happens once typing pauses, not on every character.
<code>applyEditorTextToWorkspace(...)</code>	<code>codeEditor.ts:157--189</code> — parses the trimmed textarea contents with <code>parseMiniJavaTextToWorkspaceState</code> (<code>core/parser/minijavaTextParser.ts</code> , 853 lines); on a parse error, catches <code>MiniJavaTextParseError</code> and reports it in the status line and via <code>options.onImportError</code> rather than throwing through to the caller — an in-progress, syntactically incomplete edit is an expected state, not a bug.
<code>importEditorTextToWorkspace(...)</code>	<code>codeEditor.ts:138--155</code> — the actual rebuild: sets a re-entrancy guard (§2.5), <code>workspace.clear()</code> , loads the parsed <code>Blockly.serialization.workspaces</code> state, and calls <code>workspace.cleanUp()</code> to re-lay-out top blocks.

2.4 The structural-equality guard: never delete what the user just typed

Two problems have to be solved together for a bidirectional editor to feel safe to type in: (a) the generator’s output is not necessarily byte-identical to what the user wrote (whitespace, comments,

formatting choices), and (b) a full `workspace.clear()` + rebuild on every keystroke would destroy block ids, undo history, and manual layout for a change that produces the *same program*. `codeEditor.ts` solves both with one idea: compare *parsed structure*, not text.

Function	Role
<code>workspaceStructureSignature(ws)</code>	<code>codeEditor.ts:116--122</code> — re-generates the workspace’s own code and re-parses <i>that</i> , then <code>JSON.stringify</code> s the resulting serialization state: the workspace’s canonical structural signature.
<code>editorTextMatchesCode(text,code)</code>	<code>codeEditor.ts:128--136</code> — true iff the editor’s raw text parses to the identical structure as <code>code</code> (an exact string match short-circuits first, l.129). Used by <code>syncEditorFromWorkspace</code> (<code>blocks→text</code> direction, l.107) to decide whether to overwrite the textarea at all: comment/whitespace-only edits are left alone.
<code>applyEditorTextToWorkspace’s skip check</code>	<code>codeEditor.ts:176--179</code> (<code>text→blocks</code> direction) — compares the freshly parsed state against <code>workspaceStructureSignature(workspace)</code> <i>before</i> rebuilding; on a match it reports “already matches the blocks” and returns without touching the workspace at all.

2.5 Breaking the sync feedback loop

A naive bidirectional sync is prone to an infinite (or at least user-visible flicker) loop: importing text mutates the workspace, the workspace’s change listener calls `updateCode()` (`app.ts:1122`), which calls `syncEditorFromWorkspace`, which could re-touch the very textarea that triggered the import in the first place. `suppressWorkspaceSyncUntil` (`codeEditor.ts:35`) is a simple timestamp guard: `importEditorTextToWorkspace` sets it 1500ms into the future (l.146) before mutating the workspace, and `syncEditorFromWorkspace` bails out immediately (l.100) while “now” is still before that deadline. Combined with the `document.activeElement === editor` check (§2), this means: while the user is typing, nothing else may touch the textarea; immediately after an import, nothing may touch it either, until the workspace’s own settling mutations (autosave, required-block enforcement) have had time to finish.

2.6 Other editor entry points

`EditableMiniJavaCodeEditor`’s public surface (`codeEditor.ts:21--27`) exposes `loadText` in addition to the input-driven path: called from **Open** (`app.ts:766`) to force-import a loaded `.java/.txt` file even when its structure happens to already match the current blocks (the file’s *label* must still update, l.291–302, hence `allowSkipWhenUnchanged = false` there). `resize()` is called by the shared layout coordinator (`app.ts`’s `requestLayoutResize`, cross-referenced in `ui-design-and-layout.tex`) whenever the inspector column changes width.

3 Typing

The Typing tab’s implementation — `deriveProgram` in `core/types/typeChecker.ts` and the renderer in `core/ui/typingPanel.ts` — is documented in full in `typing-and-evaluation-rules.tex` (the typing-rule inference-rule reconstruction) and was the subject of the project’s most recent

commit. This note gives only the piece that document does not: how it fits into the shared inspector-tab machinery of §1.

Symbol	Role
<code>scheduleTypingRender()</code>	<code>typingPanel.ts:163--171</code> — <code>rAF-debounced</code> ; returns immediately unless <code>#panel-typing</code> currently carries <code>is-active</code> (l.165), so a background workspace edit while another tab is open costs nothing. Called both from <code>selectInspectorPanel('typing')</code> (§1) and from the workspace’s own change listener (<code>app.ts:1109</code>) on every non-viewport mutation.
<code>initTypingPanel(workspace)</code>	<code>typingPanel.ts:173--180</code> — wires the <code>#typing-method-select</code> dropdown’s change listener and stores the workspace getter; called once from <code>app.ts:1416</code> .
<code>deriveProgram(workspace)</code>	<code>core/types/typeChecker.ts:495--497</code> — the same walk that produces the Problems tab’s diagnostics (see <code>bottom-panel-sync.tex</code>), so the two views can never disagree.

4 Outline

The Outline tab renders the block program as a plain indented tree — every top-level block, recursively, down to its structural children — each row click-to-locate to the corresponding block in the workspace. It is implemented entirely inside `app.ts` (no separate module).

4.1 Rendering

Function	Role
<code>scheduleOutlineRender()</code>	<code>app.ts:368--374</code> — <code>rAF-debounced</code> (mirrors <code>scheduleTypingRender</code> ’s pattern, though this one does not itself check tab visibility — the caller at <code>selectInspectorPanel</code> gates on the panel becoming active, and the workspace listener (<code>app.ts:1108</code>) fires it unconditionally on every edit regardless of which tab is showing, since a full outline rebuild is cheap enough not to need the same guard).
<code>renderOutline()</code>	<code>app.ts:376--427</code> — clears <code>#program-outline</code> (<code>index.html:235</code>); if the workspace has no top blocks, shows an <code>.outline-empty</code> placeholder (l.382–388); otherwise walks every top block via the local <code>addBlock(block, depth)</code> closure (l.390–424).

Function	Role
<code>addBlock(block, depth)</code>	<code>app.ts:390--424</code> (inline closure) — recursive tree walk using Blockly's own <code>block.getChildren(true)</code> (structural children, 1.391) as the recursion source — no separate outline data structure is built or maintained ; the block tree <i>is</i> the outline, walked fresh on every render. Each row is a <code>button.outline-item</code> with a CSS custom property <code>--outline-depth</code> driving indentation (1.396), a disclosure glyph (a caret if the block has children, a centered dot otherwise, 1.400–403), the block's own <code>toString(54, '...')</code> rendering truncated to 54 characters as the row label (1.407–408), and the block's type with the <code>mj_</code> prefix and underscores stripped as a small trailing tag (1.410–412).

4.2 Click-to-locate

Each row's click handler (`app.ts:415--421`) is the same three-call idiom used everywhere else in the IDE that links a rendered artifact back to its source block (Problems, CESH, A vs B — see the companion `bottom-panel-sync.tex`): `workspace.getBlockById(item.dataset.blockId)`, then `workspace.centerOnBlock(id)`, then `Blockly.common.setSelected(block)` (which, unlike `BlockSvg.select()`, also clears whatever was previously selected). The Outline row itself carries no separate "locate" affordance — the *whole row* is the button, since unlike Problems (which also needs room for a multi-line diagnostic message) an outline entry has nothing else to click.

4.3 Why Outline needs no staleness guard

Unlike the CESH/Compare/Rewrite steppers (which snapshot a `MachineState` or a copied block tree that can drift out of sync with a live-edited workspace, requiring an explicit `stale` flag — see `stepperPanel.ts:46` and the companion CESH note), the Outline and Typing tabs hold **no persistent state of their own** between renders: each call to `renderOutline/deriveProgram` reads the live workspace fresh. There is nothing to go stale, and hence no "Program changed — press Load to restart" banner in either tab, unlike the machine steppers.

5 Cross-cutting: the provenance idiom

All three tabs, and every other program-view surface in the IDE, converge on the same two- or three-line idiom to link a rendered UI element back to the block that produced it:

Tab	Locate function
Editable Code	N/A — the editor <i>is</i> a textual view of the whole program at once; there is no per-line click-to-locate, only the debounced <code>text→blocks</code> import of §2.
Typing	Every judgement button calls <code>locate(blockId)</code> (<code>typingPanel.ts:26--30</code>), which forwards to <code>locateBlock</code> from <code>typeDiagnostics.ts</code> (shared with the Problems tab, <code>typeDiagnostics.ts:37--44</code>).

Tab	Locate function
Outline	Inline in <code>addBlock</code> 's click handler (<code>app.ts:415--421</code>), using <code>centerOnBlock + Blockly.common.setSelected</code> directly.

The full inventory of this pattern across the *bottom* panel (Problems, CESK, A vs B, and the two tabs that deliberately *don't* sync back to the main workspace) is catalogued in the companion note `bottom-panel-sync.tex`.

Note on scope

This note covers the three *inspector column* tabs. The seven *bottom panel* (viz-dock) tabs — Problems, Output, Call-by-Structure, Call-by-Value, CESK, A vs B, Rewrite — are covered in `bottom-panel-sync.tex`. The Typing tab's typing-rule content is covered in `typing-and-evaluation-rules.tex`. The overall workbench shell (activity bar, perspectives, panel visibility/resizing) that the inspector column sits inside is covered in `ui-design-and-layout.tex`.